

NumPy

Python for Data Science

Prof Sara Kostan

Introduction to NumPy

- **What is NumPy?**
 - NumPy (**Numerical Python**) is a fundamental package for scientific computing. It allows:
 - Efficient storage and manipulation of numerical arrays
 - Fast mathematical operations over large datasets
- **NumPy is a numerical abstraction layer that enables efficient manipulation and computation of large numerical datasets.**

- **NumPy (Numerical Python)** is a **powerful library** that allows you to:
 - Store data efficiently using **arrays**
 - Perform **fast numerical computations**
 - Generate data for **visualization and scientific analysis**
 - It is the foundation for many other libraries in data science like Pandas, SciPy, and scikit-learn.

Why NumPy Is Used

NumPy is used to address fundamental limitations of native Python data structures in numerical contexts.

- **Limitations of Native Python Lists**

- Store elements as references, not raw numerical values
- Inefficient memory usage
- Slow execution for large-scale numerical operations
- Require explicit loops for computation

In data science and engineering, these advantages are critical for:

- ✓ Data preprocessing
- ✓ Statistical analysis
- ✓ Machine learning algorithms
- ✓ Scientific simulations

- **Advantages of NumPy**

- NumPy is used because it provides:
- **Memory efficiency** through contiguous storage
- **Computational efficiency** via vectorized operations
- **Mathematical expressiveness** with concise syntax
- **Scalability** for large datasets

NumPy enables **faster, clearer, and more mathematically expressive code.**

How NumPy Is Used

NumPy is used through an **array-based computational model**.

1. Array Creation and Representation

- Users create **ndarray** objects from:
- Python lists and tuples
- Generated numerical sequences
- Random distributions
- Mathematical constructors (zeros, ones, identity matrices)

- These arrays represent:
 1. Vectors (1D)
 2. Matrices (2D)
 3. Tensors (nD)

How NumPy Is Used

NumPy is used through an **array-based computational model**.

2. Vectorized Operations

- Operations are applied directly to arrays:
 - Arithmetic operations
 - Logical comparisons
 - Mathematical functions
 - This eliminates explicit loops and improves performance.

3. Mathematical and Statistical Processing

- NumPy provides built-in functions for:
 - Descriptive statistics
 - Aggregation
 - Linear algebra
 - Random sampling
- These operations are internally optimized and executed in compiled code.

4. Integration with the Data Science Stack

NumPy is used as:

- The **computational backend** for Pandas
- The **numerical foundation** of SciPy
- The **data container** for machine learning frameworks

Object of Study	Array-based numerical computation
Core Concept	n-dimensional array (ndarray)
Purpose	Efficient numerical and scientific computing
Method	Vectorization and optimized low-level routines
Role	Foundational layer of data science ecosystems

Installing and Importing NumPy

If not already installed

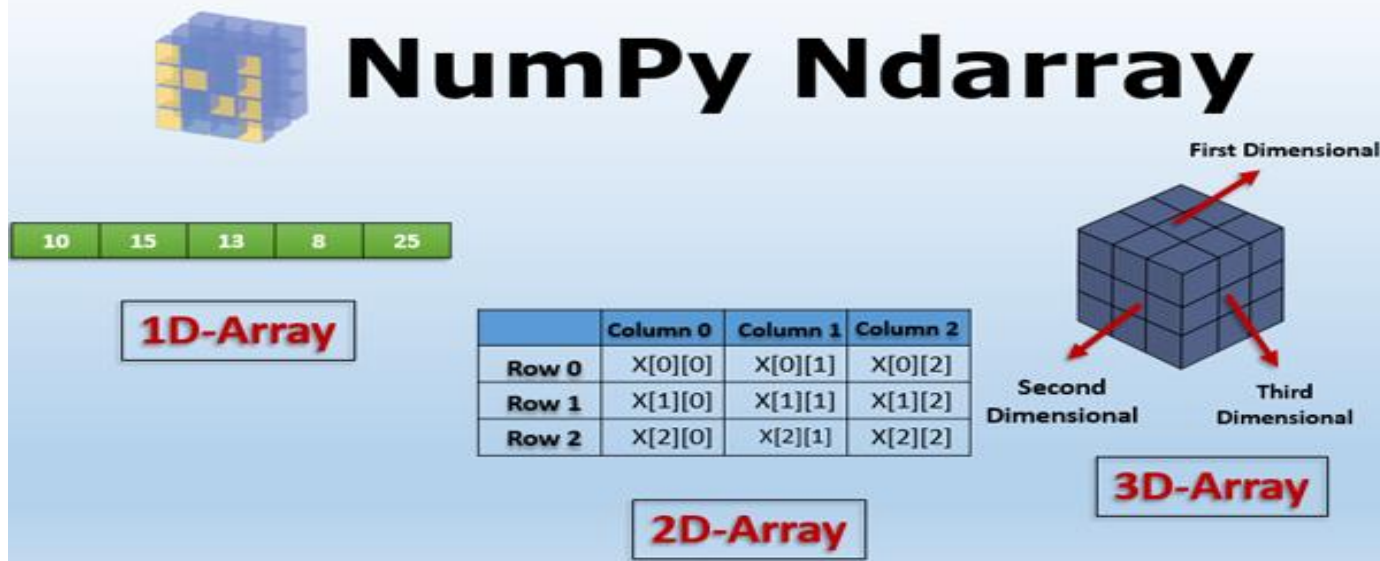
!pip install numpy

Importing

import numpy as np

NumPy ndarray (N-Dimensional Array)

- In NumPy, the core data structure is the **ndarray**, which stands for **N-dimensional array**.
- A NumPy ndarray is a **homogeneous, fixed-size, multi-dimensional** array of elements stored contiguously in memory and accessed using indices.
- This structure is optimized for **numerical computation**, making it fundamental in data science, scientific computing, and engineering applications.



Multi-Dimensional Structure

An ndarray can have:

- 1 dimension → vector
- 2 dimensions → matrix
- 3+ dimensions → tensor

`a = np.array([1, 2, 3])` # 1D

`b = np.array([[1, 2], [3, 4]])` # 2D

`c = np.zeros((2, 3, 4))` # 3D



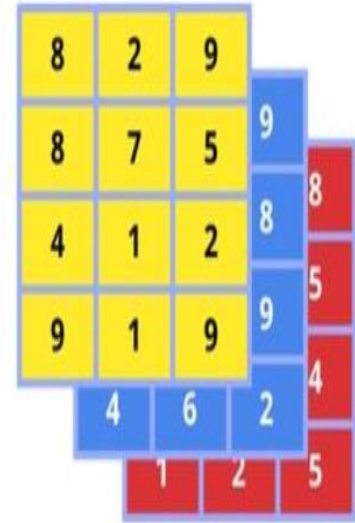
1

`my_np.array([4,5,9])`



2

`my_np.array([[8,2,9],
[8,7,5],
[4,1,2],
[9,1,9]])`



3

`my_np.array([[8,2,9],
[8,7,5],[4,1,2],[9,1,9]],
[[4,5,9],[4,2,8],
[2,1,9],[4,6,2]],
[[6,8,8],[2,4,5],
[7,8,4],[1,2,5]]])`

Creating Arrays

- From Python Lists

```
list_data = [1, 2, 3, 4, 5]
```

```
array_data = np.array(list_data)
```

```
print(array_data) # Output: [1 2 3 4 5]
```

- **Multi-dimensional (Matrix)**

```
matrix_data = np.array([[1, 2], [3, 4]])
```

```
print(matrix_data)
```

```
# Output:
```

```
# [[1 2]
```

```
# [3 4]]
```

- **Automatic Creation**

`np.zeros(5)`

`np.ones((2, 3))`

`np.arange(0, 10, 2)`

`np.linspace(0, 1, 5)`

Main NumPy ndarray Attributes

Attribute	Description	Example
ndim	Number of dimensions (axes) of the array	python arr = np.array([[1,2,3],[4,5,6]]) arr.ndim # 2
shape	Tuple representing the size of each dimension	python arr.shape # (2, 3)
size	Total number of elements in the array	python arr.size # 6
dtype	Data type of the elements	python arr.dtype # int64
itemsize	Size in bytes of each element	python arr.itemsize # 8
nbytes	Total memory consumed by the array (bytes)	python arr.nbytes # 48
T	Transpose of the array (for 2D arrays)	python arr.T
real	Real part of the array (for complex arrays)	python arr.real
imag	Imaginary part of the array (for complex arrays)	python arr.imag
flags	Information about memory layout and mutability	python arr.flags
strides	Number of bytes to step in each dimension	python arr.strides

Useful Array Creation Functions

Function	Description
<code>np.zeros((2, 3))</code>	Creates a 2x3 array of zeros
<code>np.ones((3, 3))</code>	Creates a 3x3 array of ones
<code>np.eye(3)</code>	Identity matrix (3x3)
<code>np.arange(0, 10, 2)</code>	Evenly spaced values: [0, 2, 4, 6, 8]
<code>np.linspace(0, 1, 5)</code>	5 values between 0 and 1
<code>np.random.rand(3)</code>	3 random floats in [0,1)
<code>np.random.randn(3)</code>	3 random samples from normal distribution
<code>np.random.randint(1, 10, 5)</code>	5 random integers between 1 and 10

Constructor in NumPy

- **What is a Constructor in NumPy?**
 - In object-oriented terms, a **constructor** is a function that **creates an object**. In NumPy, the primary constructor is:

```
np.array()
```

- It constructs an **ndarray object**, which is the core data structure studied in NumPy.

`np.array()` – The Fundamental Constructor

- `np.array()` converts a **Python sequence** (list, tuple) or nested sequences into a NumPy ndarray.

`np.array(object, dtype=None, ndmin=0)`

- **Key parameters**
 - object: input data (list, list of lists, tuple, etc.)
 - dtype: specifies data type
 - ndmin: forces a minimum number of dimensions
 - import numpy as np

`a = np.array([1, 2, 3], dtype=float)`

- **Built-in Array Creation Methods (Alternative Constructors)**

- Arrays with Fixed Values - **np.zeros()**

- np.zeros((2, 3))**

- Creates an array filled with zeros.

- **np.ones()** Creates an array filled with ones.

- np.ones((3, 2))**

- **np.full()** Creates an array filled with a constant.

- np.full((2, 2), 7)**

Arrays from Numerical Ranges

- **np.arange()**

np.arange(0, 10, 2)

- Similar to Python range
- Step-based spacing

- **np.linspace()**

np.linspace(0, 1, 5)

- Generates evenly spaced values
- Preferred for numerical analysis

- Random Array Constructors

- Examples

- `np.random.rand(3, 2)`

- `np.random.randn(4)`

- `np.random.randint(0, 10, size=(2, 3))`

- Used for:

- Simulations

- Sampling

- Machine learning initialization

- **Built-in Methods of the ndarray Object**
 - Once an array is created, **methods operate on the object itself.**
 - **Structural and Informational Methods**

Method	Purpose
a.shape	Dimensions
a.ndim	Number of axes
a.size	Number of elements
a.dtype	Data type

- **Transformation Methods**

- **reshape()**

```
a.reshape(3, 2)
```

- **transpose() / T**

```
a.T
```

- **Resize**

```
a.resize((3, 2))
```

- reshape → returns new view
- resize → modifies original array

- Computational Methods

- Aggregation

- `a.sum()`

- `a.mean()`

- `a.std()`

- `a.min()`

- `a.max()`

- With axis:

- `a.sum(axis=0)`

- **Logical and Comparison Methods**

`a > 5`

`a.any()`

`a.all()`

- **Built-in Universal Functions (ufuncs)**

- Vectorized functions operating element-wise on arrays.

`np.sqrt(a)`

`np.exp(a)`

`np.log(a)`

`np.abs(a)`

dtype

- dtype defines the **type** and **precision** of stored values.

`np.array([1, 2, 3], dtype=float)`

- **Importance**
 - Memory optimization
 - Numerical precision
 - Performance

Array Modification & Combination

- **Append**

`np.append([1, 2], [3, 4])`

- **hstack (Horizontal Stack)**

`np.hstack(([1, 2], [3, 4]))`

- **vstack (Vertical Stack)**

`np.vstack(([1, 2], [3, 4]))`

- **Concatenate**

`np.concatenate(([1, 2], [3, 4]))`

Array Attributes and Properties

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print("Shape:", arr.shape)      # (2, 3)
```

```
print("Size:", arr.size)        # 6 elements
```

```
print("Data type:", arr.dtype)  # int32 or int64
```

```
print("Number of dimensions:", arr.ndim) # 2
```

Indexing and Slicing

```
a = np.array([10, 20, 30, 40, 50])  
print(a[0])    # First element  
print(a[-1])   # Last element  
print(a[1:4])  # Elements from index 1 to 3  
(20,30,40)
```

- For 2D arrays:

```
b = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(b[0, 1]) # Row 0, Column 1 → 2
```

```
print(b[:, 1]) # All rows, Column 1 → [2, 5]
```

Array Operations

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
print(a + b)    # [5 7 9]
```

```
print(a * 2)    # [2 4 6]
```

```
print(a ** 2)   # [1 4 9]
```

```
print(np.dot(a, b))# Dot product: 1*4 + 2*5 + 3*6 = 32
```

Broadcasting

- Broadcasting allows NumPy to perform operations between arrays of different shapes.

```
a = np.array([1, 2, 3])
```

```
b = 2
```

```
print(a + b) # Output: [3 4 5]
```

- NumPy automatically stretches the scalar `b` to match the shape of `a`.

Common Mathematical Functions

```
x = np.array([1, 2, 3, 4])
```

```
print(np.mean(x))    # 2.5
```

```
print(np.std(x))     # Standard deviation
```

```
print(np.sum(x))     # 10
```

```
print(np.min(x))     # 1
```

```
print(np.max(x))     # 4
```


Reshaping and Flattening

```
a = np.array([[1, 2], [3, 4], [5, 6]])
```

```
print(a.shape)          # (3, 2)
```

```
reshaped = a.reshape(2, 3)
```

```
print(reshaped)          # Now a 2x3 matrix
```

```
flattened = a.flatten()
```

```
print(flattened)         # [1 2 3 4 5 6]
```

Filtering and Boolean Indexing

```
x = np.array([10, 20, 30, 40])  
filter = x > 25  
print(filter)      # [False False  True  True]  
print(x[filter])   # [30 40]
```

– Short version:

```
print(x[x > 25])   # [30 40]
```

Exercise: NumPy Practice – Student Grades Analysis

- You are given a dataset representing the grades (out of 100) of 4 students in 5 different subjects.

```
import numpy as np
grades = np.array([
    [88, 76, 92, 85, 69], # Student 1
    [78, 81, 86, 80, 73], # Student 2
    [90, 94, 89, 91, 87], # Student 3
    [65, 70, 72, 68, 60]  # Student 4
])
```

- Shape:** What is the shape of the array?
- Subject averages:** Calculate the average score for each subject.
- Student averages:** Calculate the average score for each student.
- Overall average:** What is the average score of all students in all subjects?
- Highest score:** What is the highest score and which student got it?
- Passing students:** Consider 70 as the passing mark. Create a Boolean array showing which grades are passing.
- Bonus:** Increase each student's grade by 5%, but cap the values at 100.

NUMPY REVIEW

Summary

Importing NumPy

`import numpy as np`

- `np` is a *conventional alias* used globally in academia and industry.

NumPy Array (**ndarray**)

- Core Concept: NumPy Array (**ndarray**)
 - A **NumPy array** is a homogeneous, fixed-size, multi-dimensional container of elements of the same data type.

Creating NumPy Arrays

- **From a Python List**

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
arr.shape    # dimensions  
arr.ndim     # number of dimensions  
arr.size     # total number of elements  
arr.dtype    # data type
```

- **2D Array (Matrix)**

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
```

Aspect	Python List	NumPy Array
Data type	Mixed	Homogeneous
Speed	Slower	Faster
Vector operations	Not supported	Supported
Memory	Less efficient	Optimized

Python list

```
a = [1, 2, 3]
```

```
b = [4, 5, 6]
```

$a + b \rightarrow$ concatenation

NumPy array

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

$a + b$ # element-wise addition

- **Generating Arrays Automatically**

```
np.zeros(5)  
np.ones(5)  
np.arange(0, 10, 2)  
np.linspace(0, 1, 5)
```

- **Mathematical Operations (Vectorization)**

```
arr = np.array([1, 2, 3, 4])  
  
arr + 10  
arr * 2  
arr ** 2
```

Emphasis

No loops → cleaner, faster, mathematically expressive code.

- **Basic Statistical Operations**

```
data = np.array([10, 20, 30, 40, 50])
```

```
np.mean(data)  
np.median(data)  
np.std(data)  
np.min(data)  
np.max(data)  
np.sum(data)
```

- Indexing and Slicing

```
arr = np.array([10, 20, 30, 40, 50])
```

```
arr[0]  
arr[1:4]
```

- 2D Indexing

```
matrix[0, 1]
```

- **Problem**

You have temperature measurements (°C) for 7 days.

```
temps = np.array([18, 20, 22, 21, 19, 23, 24])
```

- **Tasks**

- Compute the average temperature
- Find the maximum temperature
- Convert temperatures to Fahrenheit

- **Problem**

You have temperature measurements (°C) for 7 days.

```
temps = np.array([18, 20, 22, 21, 19, 23, 24])
```

- **Solution**

```
avg = np.mean(temps)
max_temp = np.max(temps)
fahrenheit = temps * 9/5 + 32
```

- **Tasks**

- Compute the average temperature
- Find the maximum temperature
- Convert temperatures to Fahrenheit

- **Exercise 1**

- Create a NumPy array of the first 10 positive integers.
- Compute the square of each element
- Compute the sum

- **Exercise 2 (2D Arrays)**

- Create a 3×3 matrix.
- Display its shape
- Compute the sum of all elements

- **Exercise 3**

```
scores = np.array([12, 15, 9, 18, 14])
```

- Compute the mean and standard deviation
- Increase all scores by 2 points

NumPy for Data Science

Topic	Academic Purpose	Key Concepts / Methods	Representative Example
NumPy Definition	Introduce numerical computing	High-performance arrays, vectorization	import numpy as np
ndarray	Core object of study	Homogeneous n-D array	np.array([1,2,3])
Dimensionality	Understand data structure	1D vector, 2D matrix, 3D array	np.zeros((2,3,4))
Constructors	Array creation	array, zeros, ones, arange	np.zeros((2,3))
linspace	Numerical sampling	Evenly spaced values	np.linspace(0,1,5)
dtype	Control precision & memory	int, float, bool	dtype=float
Attributes	Inspect array metadata	ndim, shape, size, dtype	a.shape
Indexing	Access elements	Indexing, slicing	a[1:4]
Slicing	Sub-array extraction	Views vs copies	a[:,0]
Shape Change	Structural manipulation	reshape, resize	a.reshape(2,3)
Append	Add elements	1D expansion	np.append(a, b)
hstack	Horizontal combination	Axis = 1	np.hstack((a,b))
vstack	Vertical combination	Axis = 0	np.vstack((a,b))
concatenate	General joining	Axis-based merge	np.concatenate((a,b))
Random	Simulations & testing	rand, randn, randint	np.random.rand(5)
Math Ops	Numerical computation	+, *, broadcasting	a*2
Aggregation	Statistics	sum, mean, std	a.mean()
Axis Ops	Multi-dim computation	Axis parameter	m.sum(axis=0)